QUBIT • ANTI-CHEAT PLATFORM

Official Problem Catalogue & Technical Reference

Complete Problem Statements, Function Signatures, Constraints, Test Cases, and Algorithmic Hints for Competitive Programming and Anti-Cheat Assessments.

93	71	21	1
TOTAL PROBLEMS	EASY TIER	MEDIUM TIER	HARD TIER

Platform Verification & Live Practice:
All 93 problems in this catalogue are active and interactive on the live QUBIT Anti-Cheat System website at:
https://rk-fatima.github.io/AntiCheatSystem/#ps

Assessment Rules & Anti-Cheat Security:
1. Every problem is evaluated against multiple hidden test cases with strict execution time limits (1.5s - 2.0s) and memory constraints.
2. Multi-tier anti-cheat monitoring monitors clipboard copy/paste, tab visibility, peripheral key switches, and AI prompt injections.
3. Point scores range from 200 pts (Easy) to 350 pts (Medium) and 500 pts (Hard) with dynamic tie-breakers based on submission timestamp.

Generated by QUBIT Anti-Cheat System | Total Catalog Score: 20,900 Points | October 15, 2026

Table of Contents: All 93 Problems

93 Problems • 20,900 PTS

#	ID	LEETCODE	PROBLEM TITLE	TIER	CATEGORY / TOPIC	PTS
1	**E1**	LC1	**Two Sum**	EASY	Arrays / Hash Table	**200**
2	**E2**	LC9	**Palindrome Number**	EASY	Math	**200**
3	**E3**	LC20	**Valid Parentheses**	EASY	Stack / Strings	**200**
4	**E4**	LC21	**Merge Two Sorted Lists**	EASY	Linked List / Two Pointers	**200**
5	**E5**	LC26	**Remove Duplicates from Sorted Array**	EASY	Arrays / Two Pointers	**200**
6	**E6**	LC27	**Remove Element**	EASY	Arrays / Two Pointers	**200**
7	**E7**	LC35	**Search Insert Position**	EASY	Binary Search	**200**
8	**E8**	LC66	**Plus One**	EASY	Arrays / Math	**200**
9	**E9**	LC67	**Add Binary**	EASY	Bit Manipulation / Strings	**200**
10	**E10**	LC83	**Remove Duplicates from Sorted List**	EASY	Linked List	**200**
11	**E11**	LC88	**Merge Sorted Array**	EASY	Arrays / Two Pointers	**200**
12	**E12**	LC94	**Binary Tree Inorder Traversal**	EASY	Trees / DFS	**200**
13	**E13**	LC100	**Same Tree**	EASY	Trees / Recursion	**200**
14	**E14**	LC101	**Symmetric Tree**	EASY	Trees	**200**
15	**E15**	LC104	**Maximum Depth of Binary Tree**	EASY	Trees / DFS	**200**
16	**E16**	LC108	**Convert Sorted Array to Binary Search Tree**	EASY	Binary Search Tree	**200**
17	**E17**	LC111	**Minimum Depth of Binary Tree**	EASY	Trees / BFS	**200**
18	**E18**	LC112	**Path Sum**	EASY	Trees / DFS	**200**
19	**E19**	LC119	**Pascal's Triangle II**	EASY	Dynamic Programming / Math	**200**
20	**E20**	LC125	**Valid Palindrome**	EASY	Strings / Two Pointers	**200**
21	**E21**	LC136	**Single Number**	EASY	Bit Manipulation	**200**
22	**E22**	LC141	**Linked List Cycle**	EASY	Linked List / Two Pointers	**200**
23	**E23**	LC144	**Binary Tree Preorder Traversal**	EASY	Trees / DFS	**200**
24	**E24**	LC145	**Binary Tree Postorder Traversal**	EASY	Trees / DFS	**200**
25	**E25**	LC160	**Intersection of Two Linked Lists**	EASY	Linked List	**200**

#	ID	LEETCODE	PROBLEM TITLE	TIER	CATEGORY / TOPIC	PTS
26	**E26**	LC169	**Majority Element**	EASY	Arrays / Boyer-Moore	**200**
27	**E27**	LC191	**Number of 1 Bits**	EASY	Bit Manipulation	**200**
28	**E28**	LC203	**Remove Linked List Elements**	EASY	Linked List	**200**
29	**E29**	LC206	**Reverse Linked List**	EASY	Linked List	**200**
30	**E30**	LC217	**Contains Duplicate**	EASY	Arrays / Hash Table	**200**
31	**E31**	LC226	**Invert Binary Tree**	EASY	Trees	**200**
32	**E32**	LC228	**Summary Ranges**	EASY	Arrays	**200**
33	**E33**	LC231	**Power of Two**	EASY	Bit Manipulation / Math	**200**
34	**E34**	LC234	**Palindrome Linked List**	EASY	Linked List / Two Pointers	**200**
35	**E35**	LC242	**Valid Anagram**	EASY	Strings / Hash Table	**200**
36	**E36**	LC257	**Binary Tree Paths**	EASY	Trees / Backtracking	**200**
37	**E37**	LC258	**Add Digits**	EASY	Math / Digital Root	**200**
38	**E38**	LC268	**Missing Number**	EASY	Arrays / Bit Manipulation	**200**
39	**E39**	LC283	**Move Zeroes**	EASY	Arrays / Two Pointers	**200**
40	**E40**	LC290	**Word Pattern**	EASY	Hash Table / Strings	**200**
41	**E41**	LC326	**Power of Three**	EASY	Math	**200**
42	**E42**	LC344	**Reverse String**	EASY	Strings / Two Pointers	**200**
43	**E43**	LC349	**Intersection of Two Arrays**	EASY	Arrays / Hash Table	**200**
44	**E44**	LC374	**Guess Number Higher or Lower**	EASY	Binary Search	**200**
45	**E45**	LC383	**Ransom Note**	EASY	Hash Table / Strings	**200**
46	**E46**	LC387	**First Unique Character in a String**	EASY	Hash Table / Strings	**200**
47	**E47**	LC392	**Is Subsequence**	EASY	Two Pointers / Strings	**200**
48	**E48**	LC412	**Fizz Buzz**	EASY	Math / Strings	**200**
49	**E49**	LC415	**Add Strings**	EASY	Math / Strings	**200**
50	**E50**	LC485	**Max Consecutive Ones**	EASY	Arrays	**200**
51	**E51**	LC507	**Perfect Number**	EASY	Math	**200**
52	**E52**	LC509	**Fibonacci Number**	EASY	Dynamic Programming / Math	**200**
53	**E53**	LC520	**Detect Capital**	EASY	Strings	**200**

#	ID	LEETCODE	PROBLEM TITLE	TIER	CATEGORY / TOPIC	PTS
54	**E54**	LC543	**Diameter of Binary Tree**	EASY	Trees / DFS	**200**
55	**E55**	LC643	**Maximum Average Subarray I**	EASY	Sliding Window	**200**
56	**E56**	LC645	**Set Mismatch**	EASY	Arrays / Hash Table	**200**
57	**E57**	LC693	**Binary Number with Alternating Bits**	EASY	Bit Manipulation	**200**
58	**E58**	LC704	**Binary Search**	EASY	Binary Search	**200**
59	**E59**	LC860	**Lemonade Change**	EASY	Greedy	**200**
60	**E60**	LC868	**Binary Gap**	EASY	Bit Manipulation	**200**
61	**E61**	LC876	**Middle of the Linked List**	EASY	Linked List / Two Pointers	**200**
62	**E62**	LC1021	**Remove Outermost Parentheses**	EASY	Stack / Strings	**200**
63	**E63**	LC1051	**Height Checker**	EASY	Arrays / Sorting	**200**
64	**E64**	LC1266	**Minimum Time Visiting All Points**	EASY	Geometry / Math	**200**
65	**E65**	LC1290	**Convert Binary Number in a Linked List to Integer**	EASY	Linked List / Bit Manipulation	**200**
66	**E66**	LC1752	**Check if Array Is Sorted and Rotated**	EASY	Arrays	**200**
67	**E67**	LC1929	**Concatenation of Array**	EASY	Arrays	**200**
68	**E68**	LC1941	**Check if All Characters Have Equal Number of Occurrences**	EASY	Strings / Hash Table	**200**
69	**E69**	LC3110	**Score of a String**	EASY	Strings	**200**
70	**E70**	LC3467	**Transform Array by Parity**	EASY	Arrays / Sorting	**200**
71	**E71**	LC3512	**Minimum Operations to Make Array Sum Divisible by K**	EASY	Arrays / Math	**200**
72	**M1**	LC2	**Add Two Numbers**	MEDIUM	Linked List / Math	**300**
73	**M2**	LC7	**Reverse Integer**	MEDIUM	Math	**300**
74	**M3**	LC11	**Container With Most Water**	MEDIUM	Two Pointers / Greedy	**300**
75	**M4**	LC19	**Remove Nth Node From End of List**	MEDIUM	Linked List / Two Pointers	**300**
76	**M5**	LC33	**Search in Rotated Sorted Array**	MEDIUM	Binary Search	**300**
77	**M6**	LC53	**Maximum Subarray**	MEDIUM	Dynamic Programming / Kadane	**300**
78	**M7**	LC75	**Sort Colors**	MEDIUM	Two Pointers / Dutch National Flag	**300**
79	**M8**	LC92	**Reverse Linked List II**	MEDIUM	Linked List	**300**

#	ID	LEETCODE	PROBLEM TITLE	TIER	CATEGORY / TOPIC	PTS
80	**M9**	LC98	**Validate Binary Search Tree**	MEDIUM	Trees / DFS	**300**
81	**M10**	LC103	**Binary Tree Zigzag Level Order Traversal**	MEDIUM	Trees / BFS	**300**
82	**M11**	LC113	**Path Sum II**	MEDIUM	Trees / Backtracking	**300**
83	**M12**	LC142	**Linked List Cycle II**	MEDIUM	Linked List / Two Pointers	**300**
84	**M13**	LC143	**Reorder List**	MEDIUM	Linked List / Two Pointers	**300**
85	**M14**	LC162	**Find Peak Element**	MEDIUM	Binary Search	**300**
86	**M15**	LC167	**Two Sum II – Input Array Is Sorted**	MEDIUM	Two Pointers / Binary Search	**300**
87	**M16**	LC189	**Rotate Array**	MEDIUM	Arrays / Two Pointers	**300**
88	**M17**	LC199	**Binary Tree Right Side View**	MEDIUM	Trees / BFS	**300**
89	**M18**	LC222	**Count Complete Tree Nodes**	MEDIUM	Trees / Binary Search	**300**
90	**M19**	LC237	**Delete Node in a Linked List**	MEDIUM	Linked List	**300**
91	**M20**	LC347	**Top K Frequent Elements**	MEDIUM	Heap / Hash Table	**300**
92	**M21**	LC2385	**Amount of Time for Binary Tree to Be Infected**	MEDIUM	Trees / BFS / Graphs	**300**
93	**H1**	LC4	**Median of Two Sorted Arrays**	HARD	Binary Search / Divide & Conquer	**400**

#1 E1 LC1 Two Sum

EASY 200 PTS

Arrays / Hash Table | ⚡ fn: twoSum()

FUNCTION SIGNATURE

```
def twoSum(nums: list[int], target: int) -> list[int]:
vector<int> twoSum(vector<int>& nums, int target)
```

PROBLEM STATEMENT

Given N integers and a target integer T, find the 0-based indices of the two numbers such that they add up to T. Print the two indices separated by a space in increasing order.

CONSTRAINTS

`2 <= N <= 10^4, -10^9 <= nums[i], T <= 10^9. Exactly one solution exists.`

EXAMPLES

Example 1

Input: `nums = [2, 7, 11, 15], target = 9`

Output: `[0, 1]`

Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].

STANDARD INPUT (STDIN):

```
4 9
2 7 11 15
```

STANDARD OUTPUT (STDOUT):

```
0 1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use a Hash Map to store elements and their indices in a single pass. For each element x, check if (target - x) already exists in the map to achieve O(N) time complexity.

#2 E2 LC9 Palindrome Number

EASY 200 PTS

Math ⚡ fn: isPalindrome()

FUNCTION SIGNATURE

```
def isPalindrome(x: int) -> bool:
bool isPalindrome(int x)
```

PROBLEM STATEMENT

Given an integer X, print true if X is a palindrome, and false otherwise.

CONSTRAINTS

`-2^31 <= X <= 2^31 - 1`

EXAMPLES

Example 1

Input: `x = 121`

Output: `true`

Explanation: 121 reads as 121 from left to right and from right to left.

STANDARD INPUT (STDIN):

```
121
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Negative numbers are never palindromes. You can check equality by comparing characters from both ends, or mathematically reverse only the second half of the digits and compare with the first half.

#3 E3 LC20 Valid Parentheses

EASY 200 PTS

Stack / Strings | ⚡ fn: isValid()

FUNCTION SIGNATURE

```
def isValid(s: str) -> bool:
bool isValid(string s)
```

PROBLEM STATEMENT

Given a string S containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid. Print true or false.

CONSTRAINTS

```
1 <= |S| <= 10^4
```

EXAMPLES

Example 1

Input: `s = "()[]{}"`

Output: `true`

Explanation: Open brackets must be closed by the same type of brackets in the correct order.

STANDARD INPUT (STDIN):

```
()[]{}
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use a Stack data structure. Push opening brackets '(', '{', '['. When encountering a closing bracket, check if the stack top matches the corresponding opening bracket and pop it. String is valid if stack is empty at the end.

#4 E4 LC21 Merge Two Sorted Lists

EASY 200 PTS

Linked List / Two Pointers | ⚡ fn: mergeTwoLists()

FUNCTION SIGNATURE

```
def mergeTwoLists(list1: Optional[ListNode], list2: Optional[ListNode]) ->
Optional[ListNode]:
ListNode* mergeTwoLists(ListNode* list1, ListNode* list2)
```

PROBLEM STATEMENT

Read N (size of first list) followed by N sorted integers. Then read M (size of second list) followed by M sorted integers. Output the merged sorted sequence separated by spaces.

CONSTRAINTS

`0 <= N, M <= 1000`

EXAMPLES

Example 1

Input: `input = 3 1 2 4 3 1 3 4`

Output: `[1, 1, 2, 3, 4, 4]`

Explanation: Merged into a single sorted list.

STANDARD INPUT (STDIN):

```
3
1 2 4
3
1 3 4
```

STANDARD OUTPUT (STDOUT):

```
1 1 2 3 4 4
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use a dummy head pointer to simplify edge cases. Iteratively compare the current node values of both lists, attaching the smaller node to the merged list and advancing its pointer.

#5 E5 LC26 Remove Duplicates from Sorted Array

EASY 200 PTS

Arrays / Two Pointers | ⚡ fn: removeDuplicates()

FUNCTION SIGNATURE

```
def removeDuplicates(nums: list[int]) -> int:
int removeDuplicates(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and a sorted integer array of size N, remove the duplicates in-place such that each unique element appears only once. Print K (number of unique elements) followed by the unique elements on the next line.

CONSTRAINTS

```
1 <= N <= 3 * 10^4
```

EXAMPLES

Example 1

Input: `nums = [1, 1, 2]`

Output: `[2, 1, 2]`

Explanation: Returns k = number of unique elements.

STANDARD INPUT (STDIN):

```
3
1 1 2
```

STANDARD OUTPUT (STDOUT):

```
2
1 2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use the Two-Pointers technique (slow and fast pointers). Since the array is sorted, whenever nums[fast] != nums[slow], advance slow and copy nums[fast] to nums[slow].

#6 E6 LC27 Remove Element

EASY 200 PTS

Arrays / Two Pointers | ⚡ fn: removeElement()

FUNCTION SIGNATURE

```
def removeElement(nums: list[int], val: int) -> int:
int removeElement(vector<int>& nums, int val)
```

PROBLEM STATEMENT

Read N and val, followed by an integer array of size N. Remove all occurrences of val. Output the count of remaining elements K, followed by the remaining elements in their relative order.

CONSTRAINTS

`0 <= N <= 100`

EXAMPLES

Example 1

Input: `nums = [3, 2, 2, 3], val = 3`

Output: `[2, 2, 2]`

Explanation: Removes all occurrences of val in-place.

STANDARD INPUT (STDIN):

```
4 3
3 2 2 3
```

STANDARD OUTPUT (STDOUT):

```
2
2 2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use Two Pointers: maintain a pointer k for valid placements. Iterate through the array; if the current element does not equal the target value, place it at index k and increment k.

#7 E7 LC35 Search Insert Position

EASY 200 PTS

Binary Search ⚡ fn: searchInsert()

FUNCTION SIGNATURE

```
def searchInsert(nums: list[int], target: int) -> int:
int searchInsert(vector<int>& nums, int target)
```

PROBLEM STATEMENT

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

CONSTRAINTS

```
1 <= N <= 10^4
```

EXAMPLES

Example 1

Input: `nums = [1, 3, 5, 6], target = 5`

Output: 2

Explanation: Return target index or insert position.

STANDARD INPUT (STDIN):

```
4 5
1 3 5 6
```

STANDARD OUTPUT (STDOUT):

```
2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use Binary Search: initialize left = 0 and right = n - 1. When nums[mid] >= target, look in the left half; otherwise look in the right half. The insertion index will be the final left pointer position.

#8 E8 LC66 Plus One

EASY 200 PTS

Arrays / Math | ⚡ fn: plusOne()

FUNCTION SIGNATURE

```
def plusOne(digits: list[int]) -> list[int]:
vector<int> plusOne(vector<int>& digits)
```

PROBLEM STATEMENT

You are given a large integer represented as an integer array of N digits. Increment the large integer by one and return the resulting array of digits separated by space.

CONSTRAINTS

```
1 <= N <= 100
```

EXAMPLES

Example 1

Input: `nums = [1, 2, 3]`
Output: `[1, 2, 4]`
Explanation: Incrementing by one gives [1,2,4].

STANDARD INPUT (STDIN):

```
3
1 2 3
```

STANDARD OUTPUT (STDOUT):

```
1 2 4
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Traverse the array from the rightmost digit to the left. If a digit is less than 9, increment it and return immediately. If it's 9, set it to 0 and continue. If all digits were 9, prepend a 1.

#9 E9 LC67 Add Binary

EASY 200 PTS

Bit Manipulation / Strings | ⚡ fn: addBinary()

FUNCTION SIGNATURE

```
def addBinary(a: str, b: str) -> str:
string addBinary(string a, string b)
```

PROBLEM STATEMENT

Given two binary strings a and b, print their sum as a binary string.

CONSTRAINTS

```
1 <= |a|, |b| <= 10^4
```

EXAMPLES

Example 1

Input: a = "11", b = "1"

Output: 100

Explanation: Binary sum of 11 and 1 is 100.

STANDARD INPUT (STDIN):

```
11 1
```

STANDARD OUTPUT (STDOUT):

```
100
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use two pointers starting at the ends of both binary strings. Keep track of a carry variable (0 or 1). Sum the bits plus carry, take sum % 2 as the new bit, and update carry = sum / 2.

#10 E10 LC83 Remove Duplicates from Sorted List

EASY 200 PTS

Linked List ⚡ fn: deleteDuplicates()

FUNCTION SIGNATURE

```
def deleteDuplicates(head: Optional[ListNode]) -> Optional[ListNode]:
ListNode* deleteDuplicates(ListNode* head)
```

PROBLEM STATEMENT

Given N and a sorted linked list of N numbers, delete all duplicates such that each element appears only once. Print the resulting sequence.

CONSTRAINTS

```
0 <= N <= 300
```

EXAMPLES

Example 1

Input: input = 3 1 1 2

Output: [1, 2]

Explanation: Remove all duplicates from sorted list.

STANDARD INPUT (STDIN):

```
3
1 1 2
```

STANDARD OUTPUT (STDOUT):

```
1 2
```

💡 ALGORITHMIC STRATEGY & INTUITION

Iterate through the linked list. While current.next exists and current.val == current.next.val, bypass the duplicate node by pointing current.next = current.next.next.

#11 E11 LC88 Merge Sorted Array

EASY 200 PTS

Arrays / Two Pointers | ⚡ fn: merge()

FUNCTION SIGNATURE

```
def merge(nums1: list[int], m: int, nums2: list[int], n: int) -> None:
void merge(vector<int>& nums1, int m, vector<int>& nums2, int n)
```

PROBLEM STATEMENT

Read m and n. Then read m integers of array nums1, followed by n integers of array nums2. Output nums1 and nums2 merged into a single sorted array.

CONSTRAINTS

```
0 <= m, n <= 200
```

EXAMPLES

Example 1

Input: input = 3 3 1 2 3 2 5 6

Output: [1, 2, 2, 3, 5, 6]

Explanation: Merge nums2 into nums1 as one sorted array.

STANDARD INPUT (STDIN):

```
3 3
1 2 3
2 5 6
```

STANDARD OUTPUT (STDOUT):

```
1 2 2 3 5 6
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Read m and n elements. Place pointers at the end of both initialized sequences and fill nums1 from back to front to avoid overwriting unmerged values in O(M+N) time.

#12 E12 LC94 Binary Tree Inorder Traversal

EASY 200 PTS

Trees / DFS | ⚡ fn: inorderTraversal()

FUNCTION SIGNATURE

```
def inorderTraversal(root: Optional[TreeNode]) -> list[int]:
vector<int> inorderTraversal(TreeNode* root)
```

PROBLEM STATEMENT

Given an array of values representing a binary tree in level-order (where 'null' represents missing child), output its inorder traversal sequence separated by space.

CONSTRAINTS

`0 <= nodes <= 100`

EXAMPLES

Example 1

Input: `input = 1 null 2 3`

Output: `[1, 3, 2]`

Explanation: Return inorder traversal of node values.

STANDARD INPUT (STDIN):

```
1 null 2 3
```

STANDARD OUTPUT (STDOUT):

```
1 3 2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

For binary tree inorder traversal (Left -> Root -> Right), use recursive DFS or an explicit stack to traverse down the leftmost path first.

#13 E13 LC100 Same Tree

EASY 200 PTS

Trees / Recursion ⚡ fn: isSameTree()

FUNCTION SIGNATURE

```
def isSameTree(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
bool isSameTree(TreeNode* p, TreeNode* q)
```

PROBLEM STATEMENT

Given level-order strings for two binary trees p and q on separate lines, check if they are identical in structure and node values. Output true or false.

CONSTRAINTS

```
0 <= nodes <= 100
```

EXAMPLES

Example 1

Input: `input = 1 2 3 1 2 3`

Output: `true`

Explanation: Return true if trees are structurally identical.

STANDARD INPUT (STDIN):

```
1 2 3
1 2 3
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use recursion or a Queue (BFS). For trees to be identical, root values must match, and the left subtrees must be identical, and the right subtrees must be identical.

#14 E14 LC101 Symmetric Tree

EASY 200 PTS

Trees ⚡ fn: isSymmetric()

FUNCTION SIGNATURE

```
def isSymmetric(root: Optional[TreeNode]) -> bool:
bool isSymmetric(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, print true if it is a mirror of itself (symmetric around its center), and false otherwise.

CONSTRAINTS

```
1 <= nodes <= 1000
```

EXAMPLES

Example 1

Input: input = 1 2 2 3 4 4 3

Output: true

Explanation: Return true if tree is symmetric around center.

STANDARD INPUT (STDIN):

```
1 2 2 3 4 4 3
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use DFS / recursion with a helper function: compare leftSubtree.left with rightSubtree.right, and leftSubtree.right with rightSubtree.left.

#15 E15 LC104 Maximum Depth of Binary Tree

EASY 200 PTS

Trees / DFS ⚡ fn: maxDepth()

FUNCTION SIGNATURE

```
def maxDepth(root: Optional[TreeNode]) -> int:
int maxDepth(TreeNode* root)
```

PROBLEM STATEMENT

Given a binary tree's level-order string, print its maximum depth (number of nodes along the longest path from root to leaf).

CONSTRAINTS

```
0 <= nodes <= 10^4
```

EXAMPLES

Example 1

Input: `input = 3 9 20 null null 15 7`

Output: 3

Explanation: Max depth from root down to furthest leaf.

STANDARD INPUT (STDIN):

```
3 9 20 null null 15 7
```

STANDARD OUTPUT (STDOUT):

```
3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use recursion: maxDepth(root) = 1 + max(maxDepth(root.left), maxDepth(root.right)). Base case: if root is null, return 0.

#16 E16 LC108 Convert Sorted Array to Binary Search Tree

EASY 200 PTS

Binary Search Tree | ⚡ fn: sortedArrayToBST()

FUNCTION SIGNATURE

```
def sortedArrayToBST(nums: list[int]) -> Optional[TreeNode]:
TreeNode* sortedArrayToBST(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and a sorted array of N numbers, convert it into a height-balanced BST. Print the root node value followed by its maximum depth.

CONSTRAINTS

```
1 <= N <= 10^4
```

EXAMPLES

Example 1

Input: input = 5 −10 −3 0 5 9

Output: [0, 3]

Explanation: Convert sorted array into height-balanced BST.

STANDARD INPUT (STDIN):

```
5
−10 −3 0 5 9
```

STANDARD OUTPUT (STDOUT):

```
0 3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

The middle element of a sorted array is the root of the balanced BST. Recursively construct the left subtree from the left half and the right subtree from the right half.

#17 E17 LC111 Minimum Depth of Binary Tree

EASY 200 PTS

Trees / BFS | ⚡ fn: minDepth()

FUNCTION SIGNATURE

```
def minDepth(root: Optional[TreeNode]) -> int:
int minDepth(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, find its minimum depth (the number of nodes along the shortest path from root to the nearest leaf).

CONSTRAINTS

`0 <= nodes <= 10^5`

EXAMPLES

Example 1

Input: `input = 3 9 20 null null 15 7`
Output: 2
Explanation: Min depth to nearest leaf node.

STANDARD INPUT (STDIN):

```
3 9 20 null null 15 7
```

STANDARD OUTPUT (STDOUT):

```
2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use dynamic programming or Fibonacci observation: ways(n) = ways(n-1) + ways(n-2). Keep two variables for previous steps to solve in O(N) time and O(1) space.

#18 E18 LC112 Path Sum

EASY 200 PTS

Trees / DFS | ⚡ fn: hasPathSum()

FUNCTION SIGNATURE

```
def hasPathSum(root: Optional[TreeNode], targetSum: int) -> bool:
bool hasPathSum(TreeNode* root, int targetSum)
```

PROBLEM STATEMENT

Read targetSum on line 1, followed by level-order representation of tree on line 2. Print true if the tree has a root-to-leaf path such that sum equals targetSum, else false.

CONSTRAINTS

```
-1000 <= targetSum <= 1000
```

EXAMPLES

Example 1

Input: input = 22 5 4 8 11 null 13 4 7 2 null null null 1

Output: true

Explanation: Check root-to-leaf path equals targetSum.

STANDARD INPUT (STDIN):

```
22
5 4 8 11 null 13 4 7 2 null null null 1
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use Two Pointers starting from both ends. Move inward skipping non-alphanumeric characters, and compare lowercase characters.

#19 E19 LC119 Pascal's Triangle II

EASY 200 PTS

Dynamic Programming / Math | ⚡ fn: getRow()

FUNCTION SIGNATURE

```
def getRow(rowIndex: int) -> list[int]:
vector<int> getRow(int rowIndex)
```

PROBLEM STATEMENT

Given an integer rowIndex (0-indexed), return the rowIndex-th row of the Pascal's triangle separated by space.

CONSTRAINTS

`0 <= rowIndex <= 33`

EXAMPLES

Example 1

Input: `input = 3`
Output: `[1, 3, 3, 1]`
Explanation: Return the kth row of Pascal triangle.

STANDARD INPUT (STDIN):

```
3
```

STANDARD OUTPUT (STDOUT):

```
1 3 3 1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use XOR bitwise property: x ^ x = 0 and x ^ 0 = x. XOR-ing all elements in the array cancels out all duplicate pairs, leaving only the single unique number in O(N) time and O(1) space.

#20 E20 LC125 Valid Palindrome

EASY 200 PTS

Strings / Two Pointers | ⚡ fn: isPalindrome()

FUNCTION SIGNATURE

```
def isPalindrome(s: str) -> bool:
bool isPalindrome(string s)
```

PROBLEM STATEMENT

A phrase is a palindrome if, after converting all uppercase letters into lowercase and removing all non-alphanumeric characters, it reads the same forward and backward. Print true or false.

CONSTRAINTS

```
1 <= |S| <= 2 * 10^5
```

EXAMPLES

Example 1

Input: `s = "A"`
Output: `true`
Explanation: Checks palindrome ignoring case and alphanumeric characters.

STANDARD INPUT (STDIN):

```
A man, a plan, a canal: Panama
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Pre-calculate digital root or use math: for non-zero numbers, the digital root is 1 + ((num - 1) % 9).

#21 E21 LC136 Single Number

EASY 200 PTS

Bit Manipulation | ⚡ fn: singleNumber()

FUNCTION SIGNATURE

```
def singleNumber(nums: list[int]) -> int:
int singleNumber(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array of N numbers where every element appears twice except for one, find and output that single element.

CONSTRAINTS

```
1 <= N <= 3 * 10^4
```

EXAMPLES

Example 1

Input: `nums = [2, 2, 1]`

Output: `1`

Explanation: Find the single element appearing once.

STANDARD INPUT (STDIN):

```
3
2 2 1
```

STANDARD OUTPUT (STDOUT):

```
1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Consider bitwise operators: XOR cancels duplicates, `n & (n - 1)` drops the lowest set bit, and shifts check bit positions. Target an optimal solution for Single Number.

#22 E22 LC141 Linked List Cycle

EASY 200 PTS

Linked List / Two Pointers | ⚡ fn: hasCycle()

FUNCTION SIGNATURE

```
def hasCycle(head: Optional[ListNode]) -> bool:
bool hasCycle(ListNode* head)
```

PROBLEM STATEMENT

Given N nodes and pos (0-indexed position where tail links back to, or -1 if no cycle), print true if there is a cycle, and false otherwise.

CONSTRAINTS

```
0 <= N <= 10^4
```

EXAMPLES

Example 1

Input: `input = 4 1 3 2 0 -4`

Output: `true`

Explanation: Detect if linked list contains a cycle.

STANDARD INPUT (STDIN):

```
4 1
3 2 0 -4
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Target an optimal solution for Linked List Cycle.

#23 E23 LC144 Binary Tree Preorder Traversal

EASY 200 PTS

Trees / DFS ⚡ fn: preorderTraversal()

FUNCTION SIGNATURE

```
def preorderTraversal(root: Optional[TreeNode]) -> list[int]:
vector<int> preorderTraversal(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, output its preorder traversal sequence separated by spaces.

CONSTRAINTS

```
0 <= nodes <= 100
```

EXAMPLES

Example 1

Input: `input = 1 null 2 3`

Output: `[1, 2, 3]`

Explanation: Preorder traversal of tree values.

STANDARD INPUT (STDIN):

```
1 null 2 3
```

STANDARD OUTPUT (STDOUT):

```
1 2 3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Depth-First Search traverses branches recursively to compute depths, validate subtrees, or aggregate path sums. Target an optimal solution for Binary Tree Preorder Traversal.

#24 E24 LC145 Binary Tree Postorder Traversal

EASY 200 PTS

Trees / DFS ⚡ fn: postorderTraversal()

FUNCTION SIGNATURE

```
def postorderTraversal(root: Optional[TreeNode]) -> list[int]:
vector<int> postorderTraversal(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, output its postorder traversal sequence separated by spaces.

CONSTRAINTS

```
0 <= nodes <= 100
```

EXAMPLES

Example 1

Input: `input = 1 null 2 3`
Output: `[3, 2, 1]`
Explanation: Postorder traversal of tree values.

STANDARD INPUT (STDIN):

```
1 null 2 3
```

STANDARD OUTPUT (STDOUT):

```
3 2 1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Depth-First Search traverses branches recursively to compute depths, validate subtrees, or aggregate path sums. Target an optimal solution for Binary Tree Postorder Traversal.

#25 E25 LC160 Intersection of Two Linked Lists

EASY 200 PTS

Linked List ⚡ fn: getIntersectionNode()

FUNCTION SIGNATURE

```
def getIntersectionNode(headA: ListNode, headB: ListNode) -> Optional[ListNode]:
ListNode* getIntersectionNode(ListNode* headA, ListNode* headB)
```

PROBLEM STATEMENT

Given two lists, find the value of the node at which the two lists intersect. If they do not intersect, print 0.

CONSTRAINTS

```
1 <= N, M <= 1000
```

EXAMPLES

Example 1

Input: input = 5 6 8 4 1 8 4 5 5 6 1 8 4 5

Output: 8

Explanation: Return the node where two lists intersect.

STANDARD INPUT (STDIN):

```
5 6 8
4 1 8 4 5
5 6 1 8 4 5
```

STANDARD OUTPUT (STDOUT):

```
8
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Focus on edge cases for Intersection of Two Linked Lists.

#26 E26 LC169 Majority Element

EASY 200 PTS

Arrays / Boyer-Moore | ⚡ fn: majorityElement()

FUNCTION SIGNATURE

```
def majorityElement(nums: list[int]) -> int:
int majorityElement(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array of N numbers, print the majority element that appears more than ⌊N / 2⌋ times.

CONSTRAINTS

```
1 <= N <= 5 * 10^4
```

EXAMPLES

Example 1

Input: `nums = [3, 2, 3]`

Output: 3

Explanation: Element that appears more than n/2 times.

STANDARD INPUT (STDIN):

```
3
3 2 3
```

STANDARD OUTPUT (STDOUT):

```
3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Majority Element.

#27 E27 LC191 **Number of 1 Bits** EASY 200 PTS

Bit Manipulation | ⚡ fn: hammingWeight()

FUNCTION SIGNATURE

```
def hammingWeight(n: int) -> int:
int hammingWeight(int n)
```

PROBLEM STATEMENT

Given a positive integer N, return the number of set bits ('1's) it has (the Hamming weight).

CONSTRAINTS

```
1 <= N <= 2^31 - 1
```

EXAMPLES

Example 1

Input: x = 11
Output: 3
Explanation: Number of set 1 bits in integer.

STANDARD INPUT (STDIN):

11

STANDARD OUTPUT (STDOUT):

3

💡 **ALGORITHMIC STRATEGY & INTUITION**

Consider bitwise operators: XOR cancels duplicates, `n & (n - 1)` drops the lowest set bit, and shifts check bit positions. Target an optimal solution for Number of 1 Bits.

#28 E28 LC203 Remove Linked List Elements

EASY 200 PTS

Linked List ⚡ fn: removeElements()

FUNCTION SIGNATURE

```
def removeElements(head: Optional[ListNode], val: int) -> Optional[ListNode]:
ListNode* removeElements(ListNode* head, int val)
```

PROBLEM STATEMENT

Read N and val on line 1, followed by N integers on line 2. Remove all nodes of value val and print the remaining list, or 'empty' if list is empty.

CONSTRAINTS

`0 <= N <= 10^4`

EXAMPLES

Example 1

Input: `input = 7 6 1 2 6 3 4 5 6`

Output: `[1, 2, 3, 4, 5]`

Explanation: Remove all nodes equal to val.

STANDARD INPUT (STDIN):

```
7 6
1 2 6 3 4 5 6
```

STANDARD OUTPUT (STDOUT):

```
1 2 3 4 5
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Focus on edge cases for Remove Linked List Elements.

#29 E29 LC206 Reverse Linked List

EASY 200 PTS

Linked List ⚡ fn: reverseList()

FUNCTION SIGNATURE

```
def reverseList(head: Optional[ListNode]) -> Optional[ListNode]:
ListNode* reverseList(ListNode* head)
```

PROBLEM STATEMENT

Given N and an integer sequence of N numbers, reverse the list and print the numbers separated by spaces.

CONSTRAINTS

`0 <= N <= 5000`

EXAMPLES

Example 1

Input: `input = 5 1 2 3 4 5`

Output: `[5, 4, 3, 2, 1]`

Explanation: Reverse singly linked list.

STANDARD INPUT (STDIN):

```
5
1 2 3 4 5
```

STANDARD OUTPUT (STDOUT):

```
5 4 3 2 1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Focus on edge cases for Reverse Linked List.

#30 E30 LC217 Contains Duplicate

EASY 200 PTS

Arrays / Hash Table | ⚡ fn: containsDuplicate()

FUNCTION SIGNATURE

```
def containsDuplicate(nums: list[int]) -> bool:
bool containsDuplicate(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array of N integers, print true if any value appears at least twice, and false if every element is distinct.

CONSTRAINTS

```
1 <= N <= 10^5
```

EXAMPLES

Example 1

Input: `nums = [1, 2, 3, 1]`

Output: `true`

Explanation: Return true if any value appears at least twice.

STANDARD INPUT (STDIN):

```
4
1 2 3 1
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Target an optimal solution for Contains Duplicate.

#31 E31 LC226 Invert Binary Tree

EASY 200 PTS

Trees | ⚡ fn: invertTree()

FUNCTION SIGNATURE

```
def invertTree(root: Optional[TreeNode]) -> Optional[TreeNode]:
TreeNode* invertTree(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, invert the tree (swap left and right child at each node) and print the resulting level-order string.

CONSTRAINTS

```
0 <= nodes <= 100
```

EXAMPLES

Example 1

Input: `input = 4 2 7 1 3 6 9`

Output: `[4, 7, 2, 9, 6, 3, 1]`

Explanation: Invert binary tree left and right children.

STANDARD INPUT (STDIN):

```
4 2 7 1 3 6 9
```

STANDARD OUTPUT (STDOUT):

```
4 7 2 9 6 3 1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Tree problems usually reduce to recursive DFS (pre-order, in-order, post-order) or iterative BFS using a queue level-by-level. Target an optimal solution for Invert Binary Tree.

#32 E32 LC228 Summary Ranges

EASY 200 PTS

Arrays ⚡ fn: summaryRanges()

FUNCTION SIGNATURE

```
def summaryRanges(nums: list[int]) -> list[str]:
vector<string> summaryRanges(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and a sorted unique integer array, return the smallest sorted list of ranges that cover all numbers. Output ranges separated by space (e.g. 0->2 4->5 7).

CONSTRAINTS

`0 <= N <= 100`

EXAMPLES

Example 1

Input: input = 6 0 1 2 4 5 7

Output: [0->2, 4->5, 7]

Explanation: Summary ranges of contiguous numbers.

STANDARD INPUT (STDIN):

```
6
0 1 2 4 5 7
```

STANDARD OUTPUT (STDOUT):

```
0->2 4->5 7
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Summary Ranges.

#33 E33 LC231 Power of Two

EASY 200 PTS

Bit Manipulation / Math | `fn: isPowerOfTwo()`

FUNCTION SIGNATURE

```
def isPowerOfTwo(n: int) -> bool:
bool isPowerOfTwo(int n)
```

PROBLEM STATEMENT

Given an integer n, print true if it is a power of two, otherwise false.

CONSTRAINTS

`-2^31 <= n <= 2^31 - 1`

EXAMPLES

Example 1

Input: `x = 16`
Output: `true`
Explanation: Check if integer is power of two.

STANDARD INPUT (STDIN):

```
16
```

STANDARD OUTPUT (STDOUT):

```
true
```

ALGORITHMIC STRATEGY & INTUITION

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Focus on edge cases for Power of Two.

#34 E34 LC234 Palindrome Linked List

EASY 200 PTS

Linked List / Two Pointers | ⚡ fn: isPalindrome()

FUNCTION SIGNATURE

```
def isPalindrome(head: Optional[ListNode]) -> bool:
bool isPalindrome(ListNode* head)
```

PROBLEM STATEMENT

Given N and a sequence of N numbers, print true if the sequence is a palindrome, else false.

CONSTRAINTS

`1 <= N <= 10^5`

EXAMPLES

Example 1

Input: `input = 4 1 2 2 1`

Output: `true`

Explanation: Check if linked list is palindrome.

STANDARD INPUT (STDIN):

```
4
1 2 2 1
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Target an optimal solution for Palindrome Linked List.

#35 E35 LC242 **Valid Anagram** EASY 200 PTS

Strings / Hash Table ⚡ fn: isAnagram()

FUNCTION SIGNATURE

```
def isAnagram(s: str, t: str) -> bool:
bool isAnagram(string s, string t)
```

PROBLEM STATEMENT

Given two lowercase strings s and t on separate lines, print true if t is an anagram of s, and false otherwise.

CONSTRAINTS

```
1 <= |s|, |t| <= 5 * 10^4
```

EXAMPLES

Example 1

Input: `a = "anagram", b = "nagaram"`
Output: `true`
Explanation: Return true if t is anagram of s.

STANDARD INPUT (STDIN):

```
anagram
nagaram
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Valid Anagram: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#36 E36 LC257 Binary Tree Paths

EASY 200 PTS

Trees / Backtracking | ⚡ fn: binaryTreePaths()

FUNCTION SIGNATURE

```
def binaryTreePaths(root: Optional[TreeNode]) -> list[str]:
vector<string> binaryTreePaths(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, return all root-to-leaf paths separated by space (formatted like 1->2->5).

CONSTRAINTS

- `1 <= nodes <= 100`

EXAMPLES

Example 1

Input: `input = 1 2 3 null 5`
Output: `[1->2->5, 1->3]`
Explanation: All root-to-leaf paths as strings.

STANDARD INPUT (STDIN):

```
1 2 3 null 5
```

STANDARD OUTPUT (STDOUT):

```
1->2->5 1->3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Tree problems usually reduce to recursive DFS (pre-order, in-order, post-order) or iterative BFS using a queue level-by-level. Focus on edge cases for Binary Tree Paths.

#37 E37 LC258 **Add Digits** EASY 200 PTS

Math / Digital Root ⚡ fn: addDigits()

FUNCTION SIGNATURE

```
def addDigits(num: int) -> int:
int addDigits(int num)
```

PROBLEM STATEMENT

Given an integer num, repeatedly add all its digits until the result has only one digit, and print it.

CONSTRAINTS

`0 <= num <= 2^31 - 1`

EXAMPLES

Example 1

Input: x = 38
Output: 2
Explanation: Repeatedly add all digits until single digit.

STANDARD INPUT (STDIN):

38

STANDARD OUTPUT (STDOUT):

2

💡 **ALGORITHMIC STRATEGY & INTUITION**

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Focus on edge cases for Add Digits.

#38 E38 LC268 Missing Number

EASY 200 PTS

Arrays / Bit Manipulation | ⚡ fn: missingNumber()

FUNCTION SIGNATURE

```
def missingNumber(nums: list[int]) -> int:
int missingNumber(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array containing N distinct numbers in the range [0, N], return the only number in the range that is missing from the array.

CONSTRAINTS

```
1 <= N <= 10^4
```

EXAMPLES

Example 1

Input: `nums = [3, 0, 1]`

Output: 2

Explanation: Return only number in range [0, n] missing.

STANDARD INPUT (STDIN):

```
3
3 0 1
```

STANDARD OUTPUT (STDOUT):

```
2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Missing Number.

#39 E39 LC283 **Move Zeroes** EASY 200 PTS

Arrays / Two Pointers | ⚡ fn: moveZeroes()

FUNCTION SIGNATURE

```
def moveZeroes(nums: list[int]) -> None:
void moveZeroes(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an integer array, move all 0's to the end while maintaining the relative order of the non-zero elements. Print the resulting array.

CONSTRAINTS

```
1 <= N <= 10^4
```

EXAMPLES

Example 1

Input: `nums = [0, 1, 0, 3, 12]`
Output: `[1, 3, 12, 0, 0]`
Explanation: Move all 0s to end while maintaining order.

STANDARD INPUT (STDIN):

```
5
0 1 0 3 12
```

STANDARD OUTPUT (STDOUT):

```
1 3 12 0 0
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Sorting allows two pointers from opposite ends or a slow/fast pointer combination to process elements in O(N). Target an optimal solution for Move Zeroes.

#40 E40 LC290 Word Pattern

EASY 200 PTS

Hash Table / Strings ⚡ fn: wordPattern()

FUNCTION SIGNATURE

```
def wordPattern(pattern: str, s: str) -> bool:
bool wordPattern(string pattern, string s)
```

PROBLEM STATEMENT

Given a pattern string and a string s of space-separated words, print true if s follows the exact same pattern bijectively, and false otherwise.

CONSTRAINTS

```
1 <= |pattern| <= 300
```

EXAMPLES

Example 1

Input: input = abba dog cat cat dog

Output: true

Explanation: Follow bijection pattern between char and word.

STANDARD INPUT (STDIN):

```
abba
dog cat cat dog
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Word Pattern: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#41 E41 LC326 Power of Three

EASY 200 PTS

Math `⚡ fn: isPowerOfThree()`

FUNCTION SIGNATURE

```
def isPowerOfThree(n: int) -> bool:
bool isPowerOfThree(int n)
```

PROBLEM STATEMENT

Given an integer n, print true if it is a power of three, else false.

CONSTRAINTS

`-2^31 <= n <= 2^31 - 1`

EXAMPLES

Example 1

Input: `x = 27`
Output: `true`
Explanation: Check if integer is power of three.

STANDARD INPUT (STDIN):

```
27
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Target an optimal solution for Power of Three.

#42 E42 LC344 Reverse String

EASY 200 PTS

Strings / Two Pointers | ⚡ fn: reverseString()

FUNCTION SIGNATURE

```
def reverseString(s: list[str]) -> None:
void reverseString(vector<char>& s)
```

PROBLEM STATEMENT

Read a string S. Print the reversed string.

CONSTRAINTS

```
1 <= |S| <= 10^5
```

EXAMPLES

Example 1

Input: `input = hello`

Output: `olleh`

Explanation: Reverse character array in-place.

STANDARD INPUT (STDIN):

```
hello
```

STANDARD OUTPUT (STDOUT):

```
olleh
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Reverse String: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#43 E43 LC349 Intersection of Two Arrays

EASY 200 PTS

Arrays / Hash Table | ⚡ fn: intersection()

FUNCTION SIGNATURE

```
def intersection(nums1: list[int], nums2: list[int]) -> list[int]:
vector<int> intersection(vector<int>& nums1, vector<int>& nums2)
```

PROBLEM STATEMENT

Read N, then N integers for array 1; then M, then M integers for array 2. Output the unique intersection elements in sorted order separated by space.

CONSTRAINTS

```
1 <= N, M <= 1000
```

EXAMPLES

Example 1

Input: input = 4 1 2 2 1 2 2 2

Output: 2

Explanation: Unique elements in both arrays.

STANDARD INPUT (STDIN):

```
4
1 2 2 1
2
2 2
```

STANDARD OUTPUT (STDOUT):

```
2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Target an optimal solution for Intersection of Two Arrays.

#44 E44 LC374 Guess Number Higher or Lower

EASY 200 PTS

Binary Search ⚡ fn: guessNumber()

FUNCTION SIGNATURE

```
def guessNumber(n: int) -> int:
int guessNumber(int n)
```

PROBLEM STATEMENT

Given n (range 1 to n) and pick (the picked secret number), find the secret number using binary search and print it.

CONSTRAINTS

```
1 <= pick <= n <= 2^31 - 1
```

EXAMPLES

Example 1

Input: `input = 10 6`

Output: `6`

Explanation: Guess number using guess API.

STANDARD INPUT (STDIN):

```
10 6
```

STANDARD OUTPUT (STDOUT):

```
6
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Identify a monotonic condition. Maintain [left, right] bounds and eliminate half the search space on each step. Target an optimal solution for Guess Number Higher or Lower.

#45 E45 LC383 Ransom Note

EASY 200 PTS

Hash Table / Strings | ⚡ fn: canConstruct()

FUNCTION SIGNATURE

```
def canConstruct(ransomNote: str, magazine: str) -> bool:
bool canConstruct(string ransomNote, string magazine)
```

PROBLEM STATEMENT

Given two strings ransomNote and magazine on separate lines, print true if ransomNote can be constructed using letters from magazine, else false.

CONSTRAINTS

```
1 <= lengths <= 10^5
```

EXAMPLES

Example 1

Input: `a = "a", b = "b"`
Output: `false`
Explanation: Construct ransomNote from letters of magazine.

STANDARD INPUT (STDIN):

```
a
b
```

STANDARD OUTPUT (STDOUT):

```
false
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Ransom Note: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#46 E46 LC387 First Unique Character in a String

EASY 200 PTS

Hash Table / Strings ⚡ fn: firstUniqChar()

FUNCTION SIGNATURE

```
def firstUniqChar(s: str) -> int:
int firstUniqChar(string s)
```

PROBLEM STATEMENT

Given a string s, find the first non-repeating character in it and print its 0-based index. If it does not exist, print -1.

CONSTRAINTS

```
1 <= |s| <= 10^5
```

EXAMPLES

Example 1

Input: `s = "leetcode"`
Output: `0`
Explanation: Find first non-repeating character index.

STANDARD INPUT (STDIN):

```
leetcode
```

STANDARD OUTPUT (STDOUT):

```
0
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for First Unique Character in a String: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#47 E47 LC392 Is Subsequence

EASY 200 PTS

Two Pointers / Strings | ⚡ fn: isSubsequence()

FUNCTION SIGNATURE

```
def isSubsequence(s: str, t: str) -> bool:
bool isSubsequence(string s, string t)
```

PROBLEM STATEMENT

Given two strings s and t on separate lines, print true if s is a subsequence of t, or false otherwise.

CONSTRAINTS

```
0 <= |s| <= 100, 0 <= |t| <= 10^4
```

EXAMPLES

Example 1

Input: a = "abc", b = "ahbgdc"
Output: true
Explanation: Check if s is subsequence of t.

STANDARD INPUT (STDIN):

```
abc
ahbgdc
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Is Subsequence: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#48 E48 LC412 Fizz Buzz

EASY 200 PTS

Math / Strings ⚡ fn: fizzBuzz()

FUNCTION SIGNATURE

```
def fizzBuzz(n: int) -> list[str]:
vector<string> fizzBuzz(int n)
```

PROBLEM STATEMENT

Given an integer n, print the 1-indexed string representation from 1 to n separated by space where multiples of 3 are 'Fizz', 5 are 'Buzz', and both are 'FizzBuzz'.

CONSTRAINTS

`1 <= n <= 10^4`

EXAMPLES

Example 1

Input: `x = 5`

Output: `[1, 2, Fizz, 4, Buzz]`

Explanation: Fizz Buzz string representation from 1 to n.

STANDARD INPUT (STDIN):

```
5
```

STANDARD OUTPUT (STDOUT):

```
1 2 Fizz 4 Buzz
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Focus on edge cases for Fizz Buzz.

#49 E49 LC415 Add Strings

EASY 200 PTS

Math / Strings | ⚡ fn: addStrings()

FUNCTION SIGNATURE

```
def addStrings(num1: str, num2: str) -> str:
string addStrings(string num1, string num2)
```

PROBLEM STATEMENT

Given two non-negative integers num1 and num2 represented as strings, return the sum of num1 and num2 as a string.

CONSTRAINTS

- `1 <= lengths <= 10^4`

EXAMPLES

Example 1

Input: `a = "11", b = "123"`
Output: 134
Explanation: Sum of non-negative integers as string.

STANDARD INPUT (STDIN):

```
11 123
```

STANDARD OUTPUT (STDOUT):

```
134
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Focus on edge cases for Add Strings.

#50 E50 LC485 Max Consecutive Ones

EASY 200 PTS

Arrays ⚡ fn: findMaxConsecutiveOnes()

FUNCTION SIGNATURE

```
def findMaxConsecutiveOnes(nums: list[int]) -> int:
int findMaxConsecutiveOnes(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and a binary array of N elements, return the maximum number of consecutive 1's in the array.

CONSTRAINTS

```
1 <= N <= 10^5
```

EXAMPLES

Example 1

Input: `nums = [1, 1, 0, 1, 1, 1]`

Output: `3`

Explanation: Maximum consecutive 1s in binary array.

STANDARD INPUT (STDIN):

```
6
1 1 0 1 1 1
```

STANDARD OUTPUT (STDOUT):

```
3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Max Consecutive Ones.

#51 E51 LC507 Perfect Number

EASY 200 PTS

Math ⚡ fn: checkPerfectNumber()

FUNCTION SIGNATURE

```
def checkPerfectNumber(num: int) -> bool:
bool checkPerfectNumber(int num)
```

PROBLEM STATEMENT

A perfect number is a positive integer that is equal to the sum of its positive divisors excluding itself. Print true if num is perfect, else false.

CONSTRAINTS

```
1 <= num <= 10^8
```

EXAMPLES

Example 1

Input: x = 28
Output: true
Explanation: Number equals sum of proper divisors.

STANDARD INPUT (STDIN):

```
28
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Target an optimal solution for Perfect Number.

#52 E52 LC509 Fibonacci Number

EASY 200 PTS

Dynamic Programming / Math ⚡ fn: fib()

FUNCTION SIGNATURE

```
def fib(n: int) -> int:
int fib(int n)
```

PROBLEM STATEMENT

Given n, calculate F(n) where F(0) = 0, F(1) = 1, and F(n) = F(n-1) + F(n-2).

CONSTRAINTS

`0 <= n <= 30`

EXAMPLES

Example 1

Input: x = 4
Output: 3
Explanation: Compute nth Fibonacci number.

STANDARD INPUT (STDIN):

4

STANDARD OUTPUT (STDOUT):

3

💡 **ALGORITHMIC STRATEGY & INTUITION**

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Focus on edge cases for Fibonacci Number.

#53 E53 LC520 Detect Capital

EASY 200 PTS

Strings ⚡ fn: detectCapitalUse()

FUNCTION SIGNATURE

```
def detectCapitalUse(word: str) -> bool:
bool detectCapitalUse(string word)
```

PROBLEM STATEMENT

Given a word S, return true if the usage of capitals in it is right (all uppercase, all lowercase, or only first letter capitalized), else false.

CONSTRAINTS

```
1 <= |S| <= 100
```

EXAMPLES

Example 1

Input: input = USA
Output: true
Explanation: Valid usage of capitals in word.

STANDARD INPUT (STDIN):

```
USA
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Detect Capital: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#54 E54 LC543 Diameter of Binary Tree

EASY 200 PTS

Trees / DFS | ⚡ fn: diameterOfBinaryTree()

FUNCTION SIGNATURE

```
def diameterOfBinaryTree(root: Optional[TreeNode]) -> int:
int diameterOfBinaryTree(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, return the length of the diameter (longest path between any two nodes in number of edges).

CONSTRAINTS

```
1 <= nodes <= 10^4
```

EXAMPLES

Example 1

Input: input = 1 2 3 4 5

Output: 3

Explanation: Longest path between any two nodes in tree.

STANDARD INPUT (STDIN):

```
1 2 3 4 5
```

STANDARD OUTPUT (STDOUT):

```
3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Depth-First Search traverses branches recursively to compute depths, validate subtrees, or aggregate path sums. Target an optimal solution for Diameter of Binary Tree.

#55 E55 LC643 **Maximum Average Subarray I** EASY 200 PTS

Sliding Window ⚡ fn: findMaxAverage()

FUNCTION SIGNATURE

```
def findMaxAverage(nums: list[int], k: int) -> float:
double findMaxAverage(vector<int>& nums, int k)
```

PROBLEM STATEMENT

Read N and k on line 1, followed by N integers on line 2. Find a contiguous subarray of length k that has the maximum average value, and print it with up to 5 decimal places.

CONSTRAINTS

```
1 <= k <= N <= 10^5
```

EXAMPLES

Example 1

Input: input = 6 4 1 12 −5 −6 50 3

Output: 12.75

Explanation: Contiguous subarray of length k with max average.

STANDARD INPUT (STDIN):

```
6 4
1 12 −5 −6 50 3
```

STANDARD OUTPUT (STDOUT):

```
12.75
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Expand the window with a right pointer while valid; shrink from the left pointer when a constraint is violated. Target an optimal solution for Maximum Average Subarray I.

#56 E56 LC645 Set Mismatch

EASY 200 PTS

Arrays / Hash Table | ⚡ fn: findErrorNums()

FUNCTION SIGNATURE

```
def findErrorNums(nums: list[int]) -> list[int]:
vector<int> findErrorNums(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array representing numbers from 1 to N with one number duplicated and one missing, output the duplicated number followed by the missing number separated by space.

CONSTRAINTS

```
2 <= N <= 10^4
```

EXAMPLES

Example 1

Input: `input = 4 1 2 2 4`

Output: `[2, 3]`

Explanation: Find duplicate and missing numbers.

STANDARD INPUT (STDIN):

```
4
1 2 2 4
```

STANDARD OUTPUT (STDOUT):

```
2 3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Target an optimal solution for Set Mismatch.

#57 E57 LC693 Binary Number with Alternating Bits

EASY 200 PTS

Bit Manipulation | ⚡ fn: hasAlternatingBits()

FUNCTION SIGNATURE

```
def hasAlternatingBits(n: int) -> bool:
bool hasAlternatingBits(int n)
```

PROBLEM STATEMENT

Given a positive integer n, check whether two adjacent bits in its binary representation always have different values. Print true or false.

CONSTRAINTS

```
1 <= n <= 2^31 - 1
```

EXAMPLES

Example 1

Input: `input = 5`
Output: `true`
Explanation: Adjacent bits always have different values.

STANDARD INPUT (STDIN):

```
5
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Consider bitwise operators: XOR cancels duplicates, `n & (n - 1)` drops the lowest set bit, and shifts check bit positions. Target an optimal solution for Binary Number with Alternating Bits.

#58 E58 LC704 Binary Search

EASY 200 PTS

Binary Search ⚡ fn: search()

FUNCTION SIGNATURE

```
def search(nums: list[int], target: int) -> int:
int search(vector<int>& nums, int target)
```

PROBLEM STATEMENT

Read N and target on line 1, followed by N sorted distinct integers. Return the 0-based index of target, or -1 if it does not exist.

CONSTRAINTS

```
1 <= N <= 10^4
```

EXAMPLES

Example 1

Input: nums = [-1, 0, 3, 5, 9, 12], target = 9

Output: 4

Explanation: Search target in sorted array in O(log n).

STANDARD INPUT (STDIN):

```
6 9
-1 0 3 5 9 12
```

STANDARD OUTPUT (STDOUT):

```
4
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Identify a monotonic condition. Maintain [left, right] bounds and eliminate half the search space on each step. Target an optimal solution for Binary Search.

#59 E59 LC860 Lemonade Change

EASY 200 PTS

Greedy | fn: lemonadeChange()

FUNCTION SIGNATURE

```
def lemonadeChange(bills: list[int]) -> bool:
bool lemonadeChange(vector<int>& bills)
```

PROBLEM STATEMENT

Each lemonade costs $5. Customers pay with $5, $10, or $20 bills. Given N and the sequence of bills, print true if you can provide every customer with correct change, else false.

CONSTRAINTS

```
1 <= N <= 10^5
```

EXAMPLES

Example 1

Input: input = 5 5 5 5 10 20

Output: true

Explanation: Provide correct change to each customer.

STANDARD INPUT (STDIN):

```
5
5 5 5 10 20
```

STANDARD OUTPUT (STDOUT):

```
true
```

ALGORITHMIC STRATEGY & INTUITION

Make the locally optimal choice at each step. Verify whether sorting the input enables a greedy strategy. Target an optimal solution for Lemonade Change.

#60 E60 LC868 Binary Gap

EASY 200 PTS

Bit Manipulation | ⚡ fn: binaryGap()

FUNCTION SIGNATURE

```
def binaryGap(n: int) -> int:
int binaryGap(int n)
```

PROBLEM STATEMENT

Given a positive integer n, find and return the longest distance between any two adjacent 1's in the binary representation of n. If no two 1's exist, print 0.

CONSTRAINTS

```
1 <= n <= 10^9
```

EXAMPLES

Example 1

Input: x = 22
Output: 2
Explanation: Longest distance between consecutive 1s in binary.

STANDARD INPUT (STDIN):

22

STANDARD OUTPUT (STDOUT):

2

💡 **ALGORITHMIC STRATEGY & INTUITION**

Consider bitwise operators: XOR cancels duplicates, `n & (n - 1)` drops the lowest set bit, and shifts check bit positions. Target an optimal solution for Binary Gap.

#61 E61 LC876 Middle of the Linked List

EASY 200 PTS

Linked List / Two Pointers | `fn: middleNode()`

FUNCTION SIGNATURE

```
def middleNode(head: Optional[ListNode]) -> Optional[ListNode]:
ListNode* middleNode(ListNode* head)
```

PROBLEM STATEMENT

Given N and a linked list of N numbers, output the list starting from the middle node to the end. (If two middle nodes, pick the second).

CONSTRAINTS

```
1 <= N <= 100
```

EXAMPLES

Example 1

Input: `input = 5 1 2 3 4 5`

Output: `[3, 4, 5]`

Explanation: Return middle node of linked list.

STANDARD INPUT (STDIN):

```
5
1 2 3 4 5
```

STANDARD OUTPUT (STDOUT):

```
3 4 5
```

ALGORITHMIC STRATEGY & INTUITION

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Target an optimal solution for Middle of the Linked List.

#62 E62 LC1021 Remove Outermost Parentheses

EASY 200 PTS

Stack / Strings | ⚡ fn: removeOuterParentheses()

FUNCTION SIGNATURE

```
def removeOuterParentheses(s: str) -> str:
string removeOuterParentheses(string s)
```

PROBLEM STATEMENT

Given a valid parentheses string S, split it into primitive valid strings, remove the outermost parentheses of each primitive string, and output the result.

CONSTRAINTS

```
1 <= |S| <= 10^5
```

EXAMPLES

Example 1

Input: s = "(()())(())"

Output: ()()()

Explanation: Remove outer parentheses of primitive valid components.

STANDARD INPUT (STDIN):

```
(()())(())
```

STANDARD OUTPUT (STDOUT):

```
()()()
```

💡 ALGORITHMIC STRATEGY & INTUITION

Stacks are ideal for matching pairs, evaluating expressions, and tracking monotonically increasing/decreasing elements. Target an optimal solution for Remove Outermost Parentheses.

#63 E63 LC1051 Height Checker

EASY 200 PTS

Arrays / Sorting | ⚡ fn: heightChecker()

FUNCTION SIGNATURE

```
def heightChecker(heights: list[int]) -> int:
int heightChecker(vector<int>& heights)
```

PROBLEM STATEMENT

Students are asked to stand in non-decreasing order of heights. Return the number of indices where the actual heights do not match the expected sorted order.

CONSTRAINTS

```
1 <= N <= 100
```

EXAMPLES

Example 1

Input: input = 6 1 1 4 2 1 3

Output: 3

Explanation: Indices where heights[i] != expected[i].

STANDARD INPUT (STDIN):

```
6
1 1 4 2 1 3
```

STANDARD OUTPUT (STDOUT):

```
3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Height Checker.

#64 E64 LC1266 Minimum Time Visiting All Points

EASY 200 PTS

Geometry / Math | ⚡ fn: minTimeToVisitAllPoints()

FUNCTION SIGNATURE

```
def minTimeToVisitAllPoints(points: list[list[int]]) -> int:
int minTimeToVisitAllPoints(vector<vector<int>>& points)
```

PROBLEM STATEMENT

Given N 2D points, return the minimum time in seconds to visit all points in order (moving 1 unit horizontally, vertically, or diagonally takes 1 second).

CONSTRAINTS

```
1 <= N <= 100
```

EXAMPLES

Example 1

Input: input = 3 1 1 3 4 -1 0

Output: 7

Explanation: Min seconds to visit all points in order.

STANDARD INPUT (STDIN):

```
3
1 1
3 4
-1 0
```

STANDARD OUTPUT (STDOUT):

```
7
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Look for mathematical invariants, modular arithmetic properties, or number theory patterns to solve in O(1) or O(log N). Focus on edge cases for Minimum Time Visiting All Points.

#65 E65 LC1290 EASY 200 PTS

Convert Binary Number in a Linked List to Integer

Linked List / Bit Manipulation ⚡ fn: getDecimalValue()

FUNCTION SIGNATURE

```
def getDecimalValue(head: ListNode) -> int:
int getDecimalValue(ListNode* head)
```

PROBLEM STATEMENT

Given N and the binary digits (0 or 1) of a linked list, return the decimal value of the number in the linked list.

CONSTRAINTS

```
1 <= N <= 30
```

EXAMPLES

Example 1

Input: input = 3 1 0 1
Output: 5
Explanation: Decimal value of binary linked list.

STANDARD INPUT (STDIN):

```
3
1 0 1
```

STANDARD OUTPUT (STDOUT):

```
5
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Focus on edge cases for Convert Binary Number in a Linked List to Integer.

#66 E66 LC1752 Check if Array Is Sorted and Rotated

EASY 200 PTS

Arrays | fn: check()

FUNCTION SIGNATURE

```
def check(nums: list[int]) -> bool:
bool check(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an integer array, return true if the array was originally sorted in non-decreasing order, then rotated some number of positions (including zero).

CONSTRAINTS

`1 <= N <= 100`

EXAMPLES

Example 1

Input: `nums = [3, 4, 5, 1, 2]`
Output: `true`
Explanation: Check if array was originally sorted and rotated.

STANDARD INPUT (STDIN):

```
5
3 4 5 1 2
```

STANDARD OUTPUT (STDOUT):

```
true
```

ALGORITHMIC STRATEGY & INTUITION

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Check if Array Is Sorted and Rotated.

#67 E67 LC1929 Concatenation of Array

EASY 200 PTS

Arrays | ⚡ fn: getConcatenation()

FUNCTION SIGNATURE

```
def getConcatenation(nums: list[int]) -> list[int]:
vector<int> getConcatenation(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array nums of length N, output an array ans of length 2N where ans is nums concatenated with itself.

CONSTRAINTS

- `1 <= N <= 1000`

EXAMPLES

Example 1

Input: `nums = [1, 2, 1]`
Output: `[1, 2, 1, 1, 2, 1]`
Explanation: Concatenation of array of length 2n.

STANDARD INPUT (STDIN):

```
3
1 2 1
```

STANDARD OUTPUT (STDOUT):

```
1 2 1 1 2 1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Concatenation of Array.

#68 E68 LC1941 EASY 200 PTS

Check if All Characters Have Equal Number of Occurrences

Strings / Hash Table ⚡ fn: areOccurrencesEqual()

FUNCTION SIGNATURE

```
def areOccurrencesEqual(s: str) -> bool:
bool areOccurrencesEqual(string s)
```

PROBLEM STATEMENT

Given a string s, return true if all characters that appear in s have the same number of occurrences, else false.

CONSTRAINTS

- `1 <= |s| <= 1000`

EXAMPLES

Example 1

Input: `s = "abacbc"`
Output: `true`
Explanation: Check if all characters have equal frequencies.

STANDARD INPUT (STDIN):

```
abacbc
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Check if All Characters Have Equal Number of Occurrences: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#69 E69 LC3110 Score of a String

EASY 200 PTS

Strings ⚡ fn: scoreOfString()

FUNCTION SIGNATURE

```
def scoreOfString(s: str) -> int:
int scoreOfString(string s)
```

PROBLEM STATEMENT

The score of a string is the sum of the absolute difference between the ASCII values of adjacent characters. Print the score of string s.

CONSTRAINTS

```
2 <= |s| <= 100
```

EXAMPLES

Example 1

Input: `s = "hello"`
Output: `13`
Explanation: Sum of absolute difference of adjacent ASCII codes.

STANDARD INPUT (STDIN):

```
hello
```

STANDARD OUTPUT (STDOUT):

```
13
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Score of a String: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#70 E70 LC3467 Transform Array by Parity

EASY 200 PTS

Arrays / Sorting | ⚡ fn: transformArray()

FUNCTION SIGNATURE

```
def transformArray(nums: list[int]) -> list[int]:
vector<int> transformArray(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array, replace each even number with 0 and each odd number with 1, then sort the array in non-decreasing order and print it.

CONSTRAINTS

```
1 <= N <= 100
```

EXAMPLES

Example 1

Input: `nums = [4, 3, 2, 1]`
Output: `[0, 0, 1, 1]`
Explanation: Replace even with 0 and odd with 1 then sort.

STANDARD INPUT (STDIN):

```
4
4 3 2 1
```

STANDARD OUTPUT (STDOUT):

```
0 0 1 1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Transform Array by Parity.

#71 E71 LC3512 EASY 200 PTS

Minimum Operations to Make Array Sum Divisible by K

Arrays / Math ⚡ fn: minOperations()

FUNCTION SIGNATURE

```
def minOperations(nums: list[int], k: int) -> int:
int minOperations(vector<int>& nums, int k)
```

PROBLEM STATEMENT

Given N and K, followed by N integers, you can subtract 1 from any element. Output the minimum number of operations to make the sum divisible by K.

CONSTRAINTS

`1 <= N <= 1000, 1 <= K <= 1000`

EXAMPLES

Example 1

Input: `input = 3 5 3 9 7`

Output: `4`

Explanation: Min operations to make array sum divisible by k.

STANDARD INPUT (STDIN):

```
3 5
3 9 7
```

STANDARD OUTPUT (STDOUT):

```
4
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Hash Maps provide O(1) average lookup. Store complements, frequencies, or prefix sums to avoid nested loops. Focus on edge cases for Minimum Operations to Make Array Sum Divisible by K.

#72 M1 LC2 Add Two Numbers

MEDIUM 300 PTS

Linked List / Math | ⚡ fn: addTwoNumbers()

FUNCTION SIGNATURE

```
def addTwoNumbers(l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2)
```

PROBLEM STATEMENT

You are given two non-empty linked lists representing two non-negative integers stored in reverse order. Add the two numbers and return the sum as a linked list in reverse order.

CONSTRAINTS

```
1 <= N, M <= 100
```

EXAMPLES

Example 1

Input: input = 3 3 2 4 3 5 6 4

Output: [7, 0, 8]

Explanation: 342 + 465 = 807.

STANDARD INPUT (STDIN):

```
3 3
2 4 3
5 6 4
```

STANDARD OUTPUT (STDOUT):

```
7 0 8
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Maintain a Hash Set of characters currently in your sliding window. Expand the right pointer; if a character repeats, shrink the window from the left until the duplicate is removed.

#73 M2 LC7 Reverse Integer

MEDIUM 300 PTS

Math `fn: reverse()`

FUNCTION SIGNATURE

```
def reverse(x: int) -> int:
int reverse(int x)
```

PROBLEM STATEMENT

Given a signed 32-bit integer x, return x with its digits reversed. If reversing x causes the value to go outside the 32-bit signed range [-2^31, 2^31 - 1], return 0.

CONSTRAINTS

`-2^31 <= x <= 2^31 - 1`

EXAMPLES

Example 1

Input: x = 123
Output: 321
Explanation: Reversed digits of 123 is 321.

STANDARD INPUT (STDIN):
123

STANDARD OUTPUT (STDOUT):
321

ALGORITHMIC STRATEGY & INTUITION

Use Two Pointers starting at index 0 and index N-1. The area is min(height[left], height[right]) * (right - left). Always advance the pointer with the smaller height.

#74 M3 LC11 Container With Most Water

MEDIUM 300 PTS

Two Pointers / Greedy | ⚡ fn: maxArea()

FUNCTION SIGNATURE

```
def maxArea(height: list[int]) -> int:
int maxArea(vector<int>& height)
```

PROBLEM STATEMENT

Given N and an integer array height of length N, find two lines that together with the x-axis form a container, such that the container contains the most water. Output the max area.

CONSTRAINTS

```
2 <= N <= 10^5
```

EXAMPLES

Example 1

Input: input = 9 1 8 6 2 5 4 8 3 7

Output: 49

Explanation: The max area of water the container can contain is 49.

STANDARD INPUT (STDIN):

```
9
1 8 6 2 5 4 8 3 7
```

STANDARD OUTPUT (STDOUT):

```
49
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Sort the array first. Fix the first element nums[i], then use Two Pointers (left and right) on the remaining subarray to find pairs summing to -nums[i]. Skip duplicate elements to avoid redundant triplets.

#75 M4 LC19 Remove Nth Node From End of List

MEDIUM 300 PTS

Linked List / Two Pointers | ⚡ fn: removeNthFromEnd()

FUNCTION SIGNATURE

```
def removeNthFromEnd(head: Optional[ListNode], n: int) -> Optional[ListNode]:
ListNode* removeNthFromEnd(ListNode* head, int n)
```

PROBLEM STATEMENT

Given N and k on line 1, followed by a linked list of N numbers on line 2, remove the k-th node from the end of the list and print the remaining list.

CONSTRAINTS

```
1 <= k <= N <= 30
```

EXAMPLES

Example 1

Input: input = 5 2 1 2 3 4 5

Output: [1, 2, 3, 5]

Explanation: Remove the 2nd node from end.

STANDARD INPUT (STDIN):

```
5 2
1 2 3 4 5
```

STANDARD OUTPUT (STDOUT):

```
1 2 3 5
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use Two Pointers (Dutch National Flag algorithm): keep low at beginning, high at end, and mid traversing. Swap 0s to low, 2s to high, and advance mid for 1s.

#76 M5 LC33 Search in Rotated Sorted Array

MEDIUM 300 PTS

Binary Search ⚡ fn: search()

FUNCTION SIGNATURE

```
def search(nums: list[int], target: int) -> int:
int search(vector<int>& nums, int target)
```

PROBLEM STATEMENT

Given N and target on line 1, followed by a rotated sorted array of distinct integers, return the 0-based index of target, or -1 if not in the array.

CONSTRAINTS

```
1 <= N <= 5000
```

EXAMPLES

Example 1

Input: `nums = [4, 5, 6, 7, 0, 1, 2], target = 0`

Output: `4`

Explanation: Return index of target if found, else -1.

STANDARD INPUT (STDIN):

```
7 0
4 5 6 7 0 1 2
```

STANDARD OUTPUT (STDOUT):

```
4
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use Modified Binary Search: identify which half of the array is sorted. If target falls within the sorted range, search there; otherwise search the other half.

#77 M6 LC53 Maximum Subarray

MEDIUM 300 PTS

Dynamic Programming / Kadane ⚡ fn: maxSubArray()

FUNCTION SIGNATURE

```
def maxSubArray(nums: list[int]) -> int:
int maxSubArray(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an integer array nums, find the subarray with the largest sum, and return its sum.

CONSTRAINTS

```
1 <= N <= 10^5
```

EXAMPLES

Example 1

Input: `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`

Output: 6

Explanation: The subarray with the largest sum is [4,-1,2,1] with sum 6.

STANDARD INPUT (STDIN):

```
9
-2 1 -3 4 -1 2 1 -5 4
```

STANDARD OUTPUT (STDOUT):

```
6
```

💡 ALGORITHMIC STRATEGY & INTUITION

Use Dynamic Programming: dp[i] represents minimum coins needed for amount i. For each coin c, dp[i] = min(dp[i], 1 + dp[i - c]).

#78 M7 LC75 Sort Colors

MEDIUM 300 PTS

Two Pointers / Dutch National Flag | ⚡ fn: sortColors()

FUNCTION SIGNATURE

```
def sortColors(nums: list[int]) -> None:
void sortColors(vector<int>& nums)
```

PROBLEM STATEMENT

Given N and an array nums with N objects colored red (0), white (1), or blue (2), sort them in-place so that objects of the same color are adjacent.

CONSTRAINTS

```
1 <= N <= 300
```

EXAMPLES

Example 1

Input: `nums = [2, 0, 2, 1, 1, 0]`

Output: `[0, 0, 1, 1, 2, 2]`

Explanation: Sort objects in-place so colors 0, 1, 2 are adjacent.

STANDARD INPUT (STDIN):

```
6
2 0 2 1 1 0
```

STANDARD OUTPUT (STDOUT):

```
0 0 1 1 2 2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use Dynamic Programming: dp[i] is true if s[0...i] can be segmented into dictionary words. Check all valid prefixes.

#79 M8 LC92 Reverse Linked List II

MEDIUM 300 PTS

Linked List | ⚡ fn: reverseBetween()

FUNCTION SIGNATURE

```
def reverseBetween(head: Optional[ListNode], left: int, right: int) -> Optional[ListNode]:
ListNode* reverseBetween(ListNode* head, int left, int right)
```

PROBLEM STATEMENT

Read N, left, right on line 1, then N numbers on line 2. Reverse the nodes of the list from position left to position right (1-indexed), and print the list.

CONSTRAINTS

```
1 <= left <= right <= N <= 500
```

EXAMPLES

Example 1

Input: `input = 5 2 4 1 2 3 4 5`

Output: `[1, 4, 3, 2, 5]`

Explanation: Reverse nodes from position left to right.

STANDARD INPUT (STDIN):

```
5 2 4
1 2 3 4 5
```

STANDARD OUTPUT (STDOUT):

```
1 4 3 2 5
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Focus on edge cases for Reverse Linked List II.

#80 M9 LC98 Validate Binary Search Tree

MEDIUM 300 PTS

Trees / DFS ⚡ fn: isValidBST()

FUNCTION SIGNATURE

```
def isValidBST(root: Optional[TreeNode]) -> bool:
bool isValidBST(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, print true if it is a valid binary search tree (BST), and false otherwise.

CONSTRAINTS

```
1 <= nodes <= 10^4
```

EXAMPLES

Example 1

Input: input = 2 1 3

Output: true

Explanation: Determine if it is a valid binary search tree.

STANDARD INPUT (STDIN):

```
2 1 3
```

STANDARD OUTPUT (STDOUT):

```
true
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Depth-First Search traverses branches recursively to compute depths, validate subtrees, or aggregate path sums. Target an optimal solution for Validate Binary Search Tree.

#81 M10 LC103 Binary Tree Zigzag Level Order Traversal

MEDIUM 300 PTS

Trees / BFS | ⚡ fn: zigzagLevelOrder()

FUNCTION SIGNATURE

```
def zigzagLevelOrder(root: Optional[TreeNode]) -> list[list[int]]:
vector<vector<int>> zigzagLevelOrder(TreeNode* root)
```

PROBLEM STATEMENT

Given a binary tree, return the zigzag level order traversal (alternating left-to-right and right-to-left) of its nodes' values, printing each level on a new line.

CONSTRAINTS

```
0 <= nodes <= 2000
```

EXAMPLES

Example 1

Input: input = 3 9 20 null null 15 7

Output: [3, 20, 9, 15, 7]

Explanation: Zigzag level order traversal of binary tree.

STANDARD INPUT (STDIN):

```
3 9 20 null null 15 7
```

STANDARD OUTPUT (STDOUT):

```
3
20 9
15 7
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Tree problems usually reduce to recursive DFS (pre-order, in-order, post-order) or iterative BFS using a queue level-by-level. Focus on edge cases for Binary Tree Zigzag Level Order Traversal.

#82 M11 LC113 Path Sum II

MEDIUM 300 PTS

Trees / Backtracking | ⚡ fn: pathSum()

FUNCTION SIGNATURE

```
def pathSum(root: Optional[TreeNode], targetSum: int) -> list[list[int]]:
vector<vector<int>> pathSum(TreeNode* root, int targetSum)
```

PROBLEM STATEMENT

Given targetSum and a binary tree, output all root-to-leaf paths where sum equals targetSum. Output each path on a new line.

CONSTRAINTS

`-1000 <= targetSum <= 1000`

EXAMPLES

Example 1

Input: `input = 22 5 4 8 11 null 13 4 7 2 null null 5 1`

Output: `[5, 4, 11, 2, 5, 8, 4, 5]`

Explanation: Return all root-to-leaf paths equaling targetSum.

STANDARD INPUT (STDIN):

```
22
5 4 8 11 null 13 4 7 2 null null 5 1
```

STANDARD OUTPUT (STDOUT):

```
5 4 11 2
5 8 4 5
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Tree problems usually reduce to recursive DFS (pre-order, in-order, post-order) or iterative BFS using a queue level-by-level. Focus on edge cases for Path Sum II.

#83 M12 LC142 Linked List Cycle II

MEDIUM 300 PTS

Linked List / Two Pointers | ⚡ fn: detectCycle()

FUNCTION SIGNATURE

```
def detectCycle(head: Optional[ListNode]) -> Optional[ListNode]:
ListNode* detectCycle(ListNode* head)
```

PROBLEM STATEMENT

Given a linked list with pos (where tail points to), return the 0-based node index where the cycle begins. If no cycle, output -1.

CONSTRAINTS

`0 <= N <= 10^4`

EXAMPLES

Example 1

Input: `input = 4 1 3 2 0 -4`

Output: `1`

Explanation: Return node where cycle begins.

STANDARD INPUT (STDIN):

```
4 1
3 2 0 -4
```

STANDARD OUTPUT (STDOUT):

```
1
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Target an optimal solution for Linked List Cycle II.

#84 M13 LC143 Reorder List

MEDIUM 300 PTS

Linked List / Two Pointers | ⚡ fn: reorderList()

FUNCTION SIGNATURE

```
def reorderList(head: Optional[ListNode]) -> None:
void reorderList(ListNode* head)
```

PROBLEM STATEMENT

Given N and a linked list L0 -> L1 -> ... -> Ln, reorder it to L0 -> Ln -> L1 -> Ln-1 -> L2 -> Ln-2... and print the resulting sequence.

CONSTRAINTS

```
1 <= N <= 5 * 10^4
```

EXAMPLES

Example 1

Input: `input = 4 1 2 3 4`

Output: `[1, 4, 2, 3]`

Explanation: Reorder list in-place L0 -> Ln -> L1 -> Ln-1.

STANDARD INPUT (STDIN):

```
4
1 2 3 4
```

STANDARD OUTPUT (STDOUT):

```
1 4 2 3
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Target an optimal solution for Reorder List.

#85 M14 LC162 **Find Peak Element** MEDIUM 300 PTS

Binary Search ⚡ fn: findPeakElement()

FUNCTION SIGNATURE

```
def findPeakElement(nums: list[int]) -> int:
int findPeakElement(vector<int>& nums)
```

PROBLEM STATEMENT

A peak element is an element that is strictly greater than its neighbors. Given N and nums, find any peak element and print its 0-based index.

CONSTRAINTS

```
1 <= N <= 1000
```

EXAMPLES

Example 1

Input: input = 4 1 2 3 1

Output: 2

Explanation: Find peak element strictly greater than neighbors.

STANDARD INPUT (STDIN):

```
4
1 2 3 1
```

STANDARD OUTPUT (STDOUT):

```
2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Identify a monotonic condition. Maintain [left, right] bounds and eliminate half the search space on each step. Target an optimal solution for Find Peak Element.

#86 M15 LC167 Two Sum II – Input Array Is Sorted

MEDIUM 300 PTS

Two Pointers / Binary Search | ⚡ fn: twoSum()

FUNCTION SIGNATURE

```
def twoSum(numbers: list[int], target: int) -> list[int]:
vector<int> twoSum(vector<int>& numbers, int target)
```

PROBLEM STATEMENT

Given N and target, followed by a 1-indexed sorted array of N integers, find the two numbers such that they add up to target. Output the 1-based indices.

CONSTRAINTS

```
2 <= N <= 3 * 10^4
```

EXAMPLES

Example 1

Input: `nums = [2, 7, 11, 15], target = 9`

Output: `[1, 2]`

Explanation: 1-indexed Two Sum in sorted array.

STANDARD INPUT (STDIN):

```
4 9
2 7 11 15
```

STANDARD OUTPUT (STDOUT):

```
1 2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Identify a monotonic condition. Maintain [left, right] bounds and eliminate half the search space on each step. Focus on edge cases for Two Sum II – Input Array Is Sorted.

#87 M16 LC189 Rotate Array

MEDIUM 300 PTS

Arrays / Two Pointers | ⚡ fn: rotate()

FUNCTION SIGNATURE

```
def rotate(nums: list[int], k: int) -> None:
void rotate(vector<int>& nums, int k)
```

PROBLEM STATEMENT

Given N and k on line 1, followed by N integers on line 2, rotate the array to the right by k steps. Print the rotated array.

CONSTRAINTS

`1 <= N <= 10^5, 0 <= k <= 10^5`

EXAMPLES

Example 1

Input: `input = 7 3 1 2 3 4 5 6 7`

Output: `[5, 6, 7, 1, 2, 3, 4]`

Explanation: Rotate array to right by k steps.

STANDARD INPUT (STDIN):

```
7 3
1 2 3 4 5 6 7
```

STANDARD OUTPUT (STDOUT):

```
5 6 7 1 2 3 4
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Sorting allows two pointers from opposite ends or a slow/fast pointer combination to process elements in O(N). Target an optimal solution for Rotate Array.

#88 M17 LC199 Binary Tree Right Side View

MEDIUM 300 PTS

Trees / BFS ⚡ fn: rightSideView()

FUNCTION SIGNATURE

```
def rightSideView(root: Optional[TreeNode]) -> list[int]:
vector<int> rightSideView(TreeNode* root)
```

PROBLEM STATEMENT

Given the level-order representation of a binary tree, imagine yourself standing on the right side of it; return the values of the nodes you can see ordered from top to bottom.

CONSTRAINTS

```
0 <= nodes <= 100
```

EXAMPLES

Example 1

Input: input = 1 2 3 null 5 null 4

Output: [1, 3, 4]

Explanation: Values visible from right side of tree.

STANDARD INPUT (STDIN):

```
1 2 3 null 5 null 4
```

STANDARD OUTPUT (STDOUT):

```
1 3 4
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Tree problems usually reduce to recursive DFS (pre-order, in-order, post-order) or iterative BFS using a queue level-by-level. Focus on edge cases for Binary Tree Right Side View.

#89 M18 LC222 Count Complete Tree Nodes

MEDIUM 300 PTS

Trees / Binary Search | ⚡ fn: countNodes()

FUNCTION SIGNATURE

```
def countNodes(root: Optional[TreeNode]) -> int:
int countNodes(TreeNode* root)
```

PROBLEM STATEMENT

Given the root of a complete binary tree in level-order, return the total number of nodes in the tree in less than O(n) time.

CONSTRAINTS

```
0 <= nodes <= 5 * 10^4
```

EXAMPLES

Example 1

Input: input = 1 2 3 4 5 6

Output: 6

Explanation: Count total nodes in complete binary tree.

STANDARD INPUT (STDIN):

```
1 2 3 4 5 6
```

STANDARD OUTPUT (STDOUT):

```
6
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Identify a monotonic condition. Maintain [left, right] bounds and eliminate half the search space on each step. Focus on edge cases for Count Complete Tree Nodes.

#90 M19 LC237 Delete Node in a Linked List

MEDIUM 300 PTS

Linked List ⚡ fn: deleteNode()

FUNCTION SIGNATURE

```
def deleteNode(node: ListNode) -> None:
void deleteNode(ListNode* node)
```

PROBLEM STATEMENT

Read N and val on line 1, followed by N distinct integers on line 2. Delete the node with value val from the linked list and print the resulting sequence.

CONSTRAINTS

```
2 <= N <= 1000
```

EXAMPLES

Example 1

Input: `input = 4 5 4 5 1 9`

Output: `[4, 1, 9]`

Explanation: Delete node from linked list given only pointer to node.

STANDARD INPUT (STDIN):

```
4 5
4 5 1 9
```

STANDARD OUTPUT (STDOUT):

```
4 1 9
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Two pointers (fast & slow) can detect cycles, locate the middle element, or maintain fixed window intervals. Focus on edge cases for Delete Node in a Linked List.

#91 M20 LC347 Top K Frequent Elements

MEDIUM 300 PTS

Heap / Hash Table | ⚡ fn: topKFrequent()

FUNCTION SIGNATURE

```
def topKFrequent(nums: list[int], k: int) -> list[int]:
vector<int> topKFrequent(vector<int>& nums, int k)
```

PROBLEM STATEMENT

Given N and k on line 1, followed by an integer array of size N on line 2, return the k most frequent elements in descending order of frequency.

CONSTRAINTS

```
1 <= k <= N <= 10^5
```

EXAMPLES

Example 1

Input: nums = [1, 1, 1, 2, 2, 3], target = 2

Output: [1, 2]

Explanation: Return k most frequent elements.

STANDARD INPUT (STDIN):

```
6 2
1 1 1 2 2 3
```

STANDARD OUTPUT (STDOUT):

```
1 2
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Algorithmic Strategy for Top K Frequent Elements: Decompose the problem, analyze time/space constraints, and apply optimal data structures.

#92 M21 LC2385 MEDIUM 300 PTS

Amount of Time for Binary Tree to Be Infected

Trees / BFS / Graphs ⚡ fn: amountOfTime()

FUNCTION SIGNATURE

```
def amountOfTime(root: Optional[TreeNode], start: int) -> int:
int amountOfTime(TreeNode* root, int start)
```

PROBLEM STATEMENT

Read start node on line 1, followed by level-order tree on line 2. At minute 0, the node start is infected. Each minute infection spreads to adjacent nodes. Output minutes needed to infect entire tree.

CONSTRAINTS

```
1 <= nodes <= 10^5
```

EXAMPLES

Example 1

Input: input = 3 1 5 3 null 4 10 6 9 2

Output: 4

Explanation: Minutes to infect entire binary tree from start node.

STANDARD INPUT (STDIN):

```
3
1 5 3 null 4 10 6 9 2
```

STANDARD OUTPUT (STDOUT):

```
4
```

💡 ALGORITHMIC STRATEGY & INTUITION

Tree problems usually reduce to recursive DFS (pre-order, in-order, post-order) or iterative BFS using a queue level-by-level. Focus on edge cases for Amount of Time for Binary Tree to Be Infected.

#93 H1 LC4 Median of Two Sorted Arrays

HARD 400 PTS

Binary Search / Divide & Conquer | ⚡ fn: findMedianSortedArrays()

FUNCTION SIGNATURE

```
def findMedianSortedArrays(nums1: list[int], nums2: list[int]) -> float:
double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2)
```

PROBLEM STATEMENT

Given two sorted arrays nums1 of size m and nums2 of size n, return the median of the two sorted arrays with up to 5 decimal places.

CONSTRAINTS

`0 <= m, n <= 1000, 1 <= m + n <= 2000`

EXAMPLES

Example 1

Input: `input = 2 1 1 3 2`

Output: `2.00000`

Explanation: merged array = [1,2,3] and median is 2.0.

STANDARD INPUT (STDIN):

```
2 1
1 3
2
```

STANDARD OUTPUT (STDOUT):

```
2.00000
```

💡 **ALGORITHMIC STRATEGY & INTUITION**

Use Binary Search on the partition of the smaller array. Partition both arrays such that the left halves have the same number of elements as the right halves and max(left) <= min(right).